

```
// =====
// 🎲 يارى دۆمىنۆى ئۇنلآين - سىرقەر
// =====

const express = require('express');
const http = require('http');
const { Server } = require('socket.io');
const path = require('path');

const app = express();
const server = http.createServer(app);
const io = new Server(server, {
  cors: { origin: '*' }
});

app.use(express.static(path.join(__dirname, 'public')));

// ===== State Management =====
const rooms = {}; // roomId -> { players, board, boneyard, currentTurn, started,

// ===== Domino Logic =====
function createDominoSet() {
  const tiles = [];
  for (let i = 0; i <= 6; i++) {
    for (let j = i; j <= 6; j++) {
      tiles.push({ left: i, right: j, id: `${i}-${j}` });
    }
  }
  return tiles;
}

function shuffle(array) {
  for (let i = array.length - 1; i > 0; i--) {
    const j = Math.floor(Math.random() * (i + 1));
    [array[i], array[j]] = [array[j], array[i]];
  }
  return array;
}

function getBoardEnds(board) {
  if (board.length === 0) return { left: null, right: null };
  return { left: board[0].left, right: board[board.length - 1].right };
}
```

```

function canPlayTile(tile, board) {
  if (board.length === 0) return { left: true, right: true };
  const ends = getBoardEnds(board);
  return {
    left: tile.left === ends.left || tile.right === ends.left,
    right: tile.left === ends.right || tile.right === ends.right
  };
}

function playerCanPlay(hand, board) {
  if (board.length === 0) return true;
  return hand.some(tile => {
    const can = canPlayTile(tile, board);
    return can.left || can.right;
  });
}

function generateRoomCode() {
  return Math.random().toString(36).substring(2, 6).toUpperCase();
}

// ===== Game Functions =====
function startGame(roomId) {
  const room = rooms[roomId];
  if (!room || room.players.length < 2) return;

  const allTiles = shuffle(createDominoSet());
  const tilesPerPlayer = room.players.length === 2 ? 7 : 6;

  room.players.forEach(p => {
    p.hand = allTiles.splice(0, tilesPerPlayer);
  });
  room.boneyard = allTiles;
  room.board = [];
  room.started = true;
  room.winner = null;
  room.passes = 0;

  // Find player with highest double
  let startIdx = 0;
  let highestDouble = -1;
  room.players.forEach((p, idx) => {
    p.hand.forEach(tile => {
      if (tile.left === tile.right && tile.left > highestDouble) {
        highestDouble = tile.left;
        startIdx = idx;
      }
    });
  });
}

```

```

    });
  });
  room.currentTurn = startIdx;

  broadcastGameState(roomId);
}

function broadcastGameState(roomId) {
  const room = rooms[roomId];
  if (!room) return;

  room.players.forEach(player => {
    const stateForPlayer = {
      roomId,
      started: room.started,
      board: room.board,
      boneyardCount: room.boneyard.length,
      currentTurn: room.currentTurn,
      currentPlayerName: room.players[room.currentTurn]?.name,
      winner: room.winner,
      yourHand: player.hand,
      yourTurn: room.players[room.currentTurn]?.id === player.id,
      players: room.players.map(p => ({
        id: p.id,
        name: p.name,
        tilesCount: p.hand.length,
        connected: p.connected
      })))
    };
    io.to(player.id).emit('gameState', stateForPlayer);
  });
}

function checkGameEnd(roomId) {
  const room = rooms[roomId];
  if (!room) return false;

  // Check if any player has empty hand
  const winner = room.players.find(p => p.hand.length === 0);
  if (winner) {
    room.winner = { name: winner.name, reason: 'empty' };
    room.started = false;
    return true;
  }

  // Check if blocked
  const allBlocked = room.players.every(p => !playerCanPlay(p.hand, room.board));

```

```

if (allBlocked && room.boneyard.length === 0) {
  let minPips = Infinity;
  let blockWinner = null;
  room.players.forEach(p => {
    const pips = p.hand.reduce((s, t) => s + t.left + t.right, 0);
    if (pips < minPips) {
      minPips = pips;
      blockWinner = p.name;
    }
  });
  room.winner = { name: blockWinner, reason: 'block', pips: minPips };
  room.started = false;
  return true;
}
return false;
}

// ===== Socket Events =====
io.on('connection', (socket) => {
  console.log('كەسنىك پەيوەست بوو', socket.id);

  // Create new room
  socket.on('createRoom', ({ playerName }) => {
    let roomId;
    do { roomId = generateRoomCode(); } while (rooms[roomId]);

    rooms[roomId] = {
      players: [{
        id: socket.id,
        name: playerName || 'ياريزان',
        hand: [],
        connected: true,
        isHost: true
      }],
      board: [],
      boneyard: [],
      currentTurn: 0,
      started: false,
      winner: null
    };

    socket.join(roomId);
    socket.data.roomId = roomId;
    socket.emit('roomCreated', { roomId, playerId: socket.id });
    broadcastLobby(roomId);
  });
}

```

```

// Join room
socket.on('joinRoom', ({ roomId, playerName }) => {
  roomId = roomId.toUpperCase();
  const room = rooms[roomId];
  if (!room) {
    socket.emit('error', { message: 'ژووره که نییه' });
    return;
  }
  if (room.started) {
    socket.emit('error', { message: 'یاریده که دهستی پیکردووه' });
    return;
  }
  if (room.players.length >= 4) {
    socket.emit('error', { message: 'زۆرتین ۴ یاریزان' });
    return;
  }

  room.players.push({
    id: socket.id,
    name: playerName || `یاریزان ${room.players.length + 1}`,
    hand: [],
    connected: true,
    isHost: false
  });

  socket.join(roomId);
  socket.data.roomId = roomId;
  socket.emit('roomJoined', { roomId, playerId: socket.id });
  broadcastLobby(roomId);
});

// Start game (host only)
socket.on('startGame', () => {
  const roomId = socket.data.roomId;
  const room = rooms[roomId];
  if (!room) return;

  const player = room.players.find(p => p.id === socket.id);
  if (!player || !player.isHost) {
    socket.emit('error', { message: 'تهنھا هۆست دهتوانیت یاریه که دهستپیکات' });
    return;
  }
  if (room.players.length < 2) {
    socket.emit('error', { message: 'پئویستی به لانیکه م ۲ یاریزان ههیه' });
    return;
  }
  startGame(roomId);
});

```

```

});

// Play tile
socket.on('playTile', ({ tileId, side }) => {
  const roomId = socket.data.roomId;
  const room = rooms[roomId];
  if (!room || !room.started) return;

  const playerIdx = room.players.findIndex(p => p.id === socket.id);
  if (playerIdx !== room.currentTurn) {
    socket.emit('error', { message: '!نۆرەى تۆ نىيە' });
    return;
  }

  const player = room.players[playerIdx];
  const tileIdx = player.hand.findIndex(t => t.id === tileId);
  if (tileIdx === -1) return;

  const tile = { ...player.hand[tileIdx] };

  if (room.board.length === 0) {
    room.board.push(tile);
  } else {
    const ends = getBoardEnds(room.board);
    if (side === 'left') {
      if (tile.right === ends.left) {
        room.board.unshift(tile);
      } else if (tile.left === ends.left) {
        room.board.unshift({ left: tile.right, right: tile.left, id: tile.id });
      } else {
        socket.emit('error', { message: '!دۆمىنۆكە ناگونجىت' });
        return;
      }
    } else {
      if (tile.left === ends.right) {
        room.board.push(tile);
      } else if (tile.right === ends.right) {
        room.board.push({ left: tile.right, right: tile.left, id: tile.id });
      } else {
        socket.emit('error', { message: '!دۆمىنۆكە ناگونجىت' });
        return;
      }
    }
  }

  player.hand.splice(tileIdx, 1);
  room.passes = 0;

```

```

    if (!checkGameEnd(roomId)) {
      room.currentTurn = (room.currentTurn + 1) % room.players.length;
    }
    broadcastGameState(roomId);
  });

  // Draw tile
  socket.on('drawTile', () => {
    const roomId = socket.data.roomId;
    const room = rooms[roomId];
    if (!room || !room.started) return;

    const playerId = room.players.findIndex(p => p.id === socket.id);
    if (playerId !== room.currentTurn) return;
    if (room.boneyard.length === 0) {
      socket.emit('error', { message: 'هېچ دۆمىنۆيەك نەماوہ' });
      return;
    }

    const tile = room.boneyard.pop();
    room.players[playerId].hand.push(tile);
    broadcastGameState(roomId);
  });

  // Pass turn
  socket.on('passTurn', () => {
    const roomId = socket.data.roomId;
    const room = rooms[roomId];
    if (!room || !room.started) return;

    const playerId = room.players.findIndex(p => p.id === socket.id);
    if (playerId !== room.currentTurn) return;

    const player = room.players[playerId];
    if (room.boneyard.length > 0) {
      socket.emit('error', { message: 'سەرەتا دۆمىنۆ وەربگرہ' });
      return;
    }
    if (playerCanPlay(player.hand, room.board)) {
      socket.emit('error', { message: 'تۆ دۆمىنۆ گونجاوت ھەيہ' });
      return;
    }

    room.passes = (room.passes || 0) + 1;
    if (!checkGameEnd(roomId)) {
      room.currentTurn = (room.currentTurn + 1) % room.players.length;
    }
  });

```

```

    }
    broadcastGameState(roomId);
  });

  // Restart game
  socket.on('restartGame', () => {
    const roomId = socket.data.roomId;
    const room = rooms[roomId];
    if (!room) return;
    const player = room.players.find(p => p.id === socket.id);
    if (!player || !player.isHost) return;
    if (room.players.length >= 2) startGame(roomId);
  });

  // Disconnect
  socket.on('disconnect', () => {
    console.log('کده سٲک ده رچوو', socket.id);
    const roomId = socket.data.roomId;
    if (!roomId || !rooms[roomId]) return;

    const room = rooms[roomId];
    const playerId = room.players.findIndex(p => p.id === socket.id);
    if (playerId === -1) return;

    if (!room.started) {
      // Remove from lobby
      room.players.splice(playerId, 1);
      if (room.players.length === 0) {
        delete rooms[roomId];
        return;
      }
      // Reassign host if needed
      if (!room.players.some(p => p.isHost)) {
        room.players[0].isHost = true;
      }
      broadcastLobby(roomId);
    } else {
      // Mark as disconnected
      room.players[playerId].connected = false;
      broadcastGameState(roomId);
      // Delete room if all disconnected
      if (room.players.every(p => !p.connected)) {
        delete rooms[roomId];
      }
    }
  });
});

```



```
function broadcastLobby(roomId) {
  const room = rooms[roomId];
  if (!room) return;
  io.to(roomId).emit('lobbyUpdate', {
    roomId,
    players: room.players.map(p => ({
      id: p.id,
      name: p.name,
      isHost: p.isHost
    }))
  });
}

// ===== Start Server =====
const PORT = process.env.PORT || 3000;
server.listen(PORT, () => {
  console.log(`🎮 سىڭىزنىڭ يارى دۆمىنىڭ كارا بوو لەسەر پۇرتى ${PORT}`);
  console.log(`بىكەرەۋە لە: http://localhost:${PORT}`);
});
```